Program 1.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
data=pd.read_csv('House Price India.csv')
data.head()
data.isnull().sum()
Q1=data['Price'].quantile(0.25)
Q3=data['Price'].quantile(0.75)
IQR=Q3-Q1
print(f"The 25th percentile(Q1) of house price is:${Q1:,.2f}")
print(f"The 75th percentile (Q3) of house price is:${Q3:,.2f}")
print(f"The interquartile (IQR) of house price is:${IQR:,.2f}")
plt.figure(figsize=(10,6))
sns.boxplot(y=data["Price"],color='skyblue')
plt.hlines(Q1,xmin=-0.4,xmax=0.4,colors='red',linestyles='dashed',label=f'Q1(${Q1:,.0f})')
plt.hlines(Q3,xmin=-0.4,xmax=0.4,colors='green',linestyles='dashed',label=f'Q3(${Q3:,.0f})')
plt.text(-0.45,Q1,'Q1',va='center',ha='right',color='red',fontsize=8)
plt.text(-0.45,Q3,'Q3',va='center',ha='right',color='green',fontsize=8)
plt.title("Box plot of house price with Q1 and Q3")
plt.ylabel("House price ($)")
plt.grid(axis='y',linestyle="--",alpha=0.7)
plt.legend() plt.show()
```

program 2.

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

import numpy as np

df = pd.read_csv("ecommerce_customer_data.csv")

df['Satisfaction Level'] = pd.cut(df['customer_satisfaction'],bins=[1, 2, 3, 5],labels=['Low', 'Medium',
'High'])

# High satisfaction → mostly repeat customers

high = df['Satisfaction Level'] == 'High'

df.loc[high, 'Repeat Purchase'] = np.random.choice(['Yes', 'No'], size=high.sum(), p=[0.85, 0.15])

# Medium satisfaction → balanced

medium = df['Satisfaction Level'] == 'Medium'

df.loc[medium, 'Repeat Purchase'] = np.random.choice(['Yes', 'No'], size=medium.sum(), p=[0.55,
0.45])

# Low satisfaction → mostly churn

low = df['Satisfaction Level'] == 'Low'

df.loc[low, 'Repeat Purchase'] = np.random.choice(['Yes', 'No'], size=low.sum(), p=[0.25, 0.75])

# Bar Plot

sns.set_style("whitegrid")

plt.figure(figsize=(8,5))

sns.countplot(data=df,x='Satisfaction Level',hue='Repeat Purchase',palette='Set2')

plt.title("Satisfaction vs Repeat Purchase")

plt.xlabel("Satisfaction Level") plt.ylabel("Number of Customers")

plt.show()

cross = pd.crosstab(df['Satisfaction Level'], df['Repeat Purchase'])

cross.plot(kind='bar', stacked=True, colormap='viridis')

plt.title("Stacked Bar Chart")

plt.ylabel("Count") plt.show()

print("\nCrosstab:") print(cross)

percentage = pd.crosstab(df['Satisfaction Level'], df['Repeat Purchase'],normalize='index') * 100
print("\nPercentage Distribution:")    print(percentage.round(2))
```

program3

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv("Automobile_data.csv")
df.head(5)
df.columns()
df = df.drop(['index','body-style','engine-type', 'num-of-cylinders'],axis=1)
df.head(5)
df.isnull().sum()
df['price']=df['price'].fillna(df['price'].mean())
plot sns.pairplot(df,diag_kind='hist')
plt.show()
corr_matrix= df.corr(numeric_only=True)
print("Correlation matrix")
print(corr_matrix)
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix Heatmap')
plt.show()
```

program 4

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
np.random.seed(42)
population_size = 100000
population_salaries = np.random.lognormal(mean=10, sigma=0.5, size=population_size)
sample_size = 50
num_samples = 10
sample_means = []
for _ in range(num_samples):
sample = np.random.choice(population_salaries, size=sample_size, replace=False)
sample_means.append(np.mean(sample))
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
sns.histplot(population_salaries, kde=True, bins=50, color='skyblue', edgecolor='black')
plt.title("Population Salary Distribution (Skewed)")
plt.xlabel("Salary") plt.ylabel("Frequency")

plt.subplot(1,2,2)
sns.histplot(sample_means, kde=True, bins=8, color='skyblue', edgecolor='black')
plt.title("Distribution of Sample Means (n=50, 10 samples)")
plt.xlabel("Sample Mean Salary") plt.ylabel("Frequency") plt.tight_layout() plt.show()
print("Population Mean Salary:", np.mean(population_salaries))
print("Sample Means:", sample_means)
print("Mean of Sample Means:", np.mean(sample_means))
```

```
program 5
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
n = 20
sample_mean = 8.0
sample_sd = 2.0
mu0 = 0.0
alpha = 0.05
df = n – 1
se = sample_sd / np.sqrt(n)
t_stat = (sample_mean - mu0) / se
p_value = 2 * stats.t.sf(abs(t_stat), df)
t_critical = stats.t.ppf(1 - alpha / 2, df)
ci_lower = sample_mean - t_critical * se
ci_upper = sample_mean + t_critical * se
print("One-Sample t-Test")
print(f"t-statistic: {t_stat:.4f}")
print(f"Critical t-value : ±{t_critical:.4f}")
print(f"Two-tailed p-value : {p_value:.4e}")
print(f"95% Confidence Interval : ({ci_lower:.4f}, {ci_upper:.4f}) bpm")
print("\nDecision:")
if p_value < alpha:
print("Reject the null hypothesis (H0).")
print("Conclusion: The drug significantly changes the mean heart rate.")
else:
print("Fail to reject the null hypothesis (H0).")
print("Conclusion: There is insufficient evidence that the drug changes the mean heart rate.")
# Plot the t-distribution
x = np.linspace(-5, 20, 1000)
y = stats.t.pdf(x, df)
plt.figure(figsize=(10, 5))
```

```
plt.plot(x, y, label=f"t-distribution (df={df})")
plt.fill_between(x, 0, y, where=(x <= -t_critical), color='red', alpha=0.3)
plt.fill_between(x, 0, y, where=(x >= t_critical), color='red', alpha=0.3)
plt.axvline(t_stat, color='blue', linestyle='--', linewidth=2,label=f"Observed t = {t_stat:.2f}")
plt.title("One-Sample Two-Tailed t-Test")
plt.xlabel("t-value")
plt.ylabel("Probability Density")
plt.legend()    plt.grid(True)          plt.show()
```

Program 6

```
import numpy as np
from statsmodels.stats.proportion import proportions_ztest
import matplotlib.pyplot as plt
from scipy.stats import norm
sales = np.array([120, 150])
users = np.array([1000, 1200])    alpha = 0.05
rate_A = sales[0] / users[0]
rate_B = sales[1] / users[1]
z_stat, p_value = proportions_ztest(sales, users)
z_critical = norm.ppf(1 - alpha/2)
print("A/B Testing Using Two-Proportion Z-Test")
print(f"\nZ-statistic : {z_stat:.4f}")
print(f"P-value : {p_value:.4f}")
print(f"Critical Z-value : ±{z_critical:.4f}")
print("\nDecision:")
if p_value < alpha:
   print("Reject the null hypothesis (H0).")
   print("Conclusion: There is a statistically significant difference between the conversion rates.")
else:
   print("Fail to reject the null hypothesis (H0).")
   print("Conclusion: There is no statistically significant difference between the conversion rates.
x = np.linspace(-4, 4, 1000)
y = norm.pdf(x)
plt.figure(figsize=(10,5))
plt.plot(x, y, label='Standard Normal Distribution')
plt.fill_between(x, 0, y, where=(x <= -z_critical),color='red', alpha=0.3, label='Rejection Region')
plt.fill_between(x, 0, y, where=(x >= z_critical),color='red', alpha=0.3)
plt.axvline(z_stat, color='blue', linestyle='--',linewidth=2, label=f'Observed Z = {z_stat:.2f}')
plt.title("Two-Proportion Z-Test (A/B Test)")
plt.xlabel("Z-value") plt.ylabel("Density")
plt.grid(True)          plt.legend()          plt.show()
```

```
program 7
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
n1 = 30
n2 = 30
mean1 = 1000
mean2 = 950
sd1 = 100
sd2 = 120
alpha = 0.05
se = np.sqrt((sd1**2 / n1) + (sd2**2 / n2))
t_stat = (mean1 - mean2) / se
df = ((sd1**2 / n1 + sd2**2 / n2)**2) / (
((sd1**2 / n1)**2 / (n1 - 1)) + ((sd2**2 / n2)**2 / (n2 - 1)))
p_value = 2 * stats.t.sf(abs(t_stat), df)
t_critical = stats.t.ppf(1 - alpha/2, df)
difference = mean1 - mean2
ci_lower = difference - t_critical * se
ci_upper = difference + t_critical * se
print("Independent Two-Sample t-Test")
print("-" * 40)
print(f"t-statistic : {t_stat:.4f}")
print(f"Critical t-value : ±{t_critical:.4f}")
print(f"Two-tailed p-value : {p_value:.4f}")
print(f"95% Confidence Interval : (${ci_lower:.2f}, ${ci_upper:.2f})")
print("\nDecision:")
if p_value < alpha:
 print("Reject the null hypothesis (H0).")
 print("Conclusion: There is a significant difference in average daily sales.")
else:
 print("Fail to reject the null hypothesis (H0).")
```

```
 print("Conclusion: There is no significant difference in average daily sales.")
x = np.linspace(-4, 4, 1000)
y = stats.t.pdf(x, df)
plt.figure(figsize=(10,5))
plt.plot(x, y, label=f"t-distribution (df={df:.1f})")
plt.fill_between(x, 0, y, where=(x <= -t_critical),color='red', alpha=0.3)
plt.fill_between(x, 0, y, where=(x >= t_critical), color='red', alpha=0.3)
plt.axvline(t_stat, color='blue', linestyle='--',linewidth=2, label=f'Observed t = {t_stat:.2f}')
plt.title("Independent Two-Sample t-Test")
plt.xlabel("t-value")            plt.ylabel("Density")
plt.grid(True)           plt.legend()    plt.show()
```

program 8

```
import pandas as pd
from sklearn.linear_model import LinearRegression
df = pd.read_csv('employee_salary_data.csv') df.head()
df_encoded = pd.get_dummies(df, columns=['Education_Level'],dtype=int)
df_encoded.head()
X = df_encoded.drop(columns=['Employee_ID', 'Salary_INR'])
y = df_encoded['Salary_INR']
model = LinearRegression()
model.fit(X, y)
print(f"Intercept (Baseline High School Salary): {model.intercept_:.2f}")
for feature, coef in zip(X.columns, model.coef_):
   print(f"Coefficient for {feature}: {coef:+.2f}")
import seaborn as sns
sns.set_theme(style="whitegrid")
g = sns.lmplot(data=df,x="Years_Experience", y="Salary_INR", hue="Education_Level")
```

program 9

```
df=pd.read_csv('housing_prices_spline1.csv')      df.head()
importstatsmodels.api assm
from patsy import dmatrix
spline_basis = dmatrix("bs(SquareFootage, knots=(2000,), degree=1, include_intercept=False)",
{"SquareFootage": df['SquareFootage']}, return_type='dataframe')
model = sm.OLS(df['Price'], spline_basis).fit()
print(model.summary())
df['FittedPrice'] = model.predict(spline_basis)
Notes: Standard Errors assume that the covariance matrix of the errors is correctly specified.
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.scatter(df['SquareFootage'], df['Price'], facecolors='none', edgecolors='blue', label='Data')
plt.plot(df['SquareFootage'], df['FittedPrice'], color='red', label='Spline Regression Fit')
plt.axvline(x=2000, color='grey', linestyle='--', label='Knot at 2000 sq ft')
```

```
plt.xlabel('Square Footage')
plt.ylabel('Price ($)')
plt.legend()
plt.grid(True)
plt.title('Housing Prices vs. Square Footage ')
plt.show()
```

```
program 10
import math
def expected_visits(age, condition):
log_lambda = 2.5 - 0.03 * age + 0.5 * condition lambda_val = math.exp(log_lambda)
return lambda_val
age = 60
visits_with_condition = expected_visits(age, condition=1)
visits_without_condition = expected_visits(age, condition=0)
print(f"Expected visits with chronic condition: {visits_with_condition:.2f}")
print(f"Expected visits without chronic condition: {visits_without_condition:.2f}")
print(f"Increase due to condition: {(visits_with_condition / visits_without_condition):.2f}x")
```

```
program 11
from scipy importstats
import math
mu_0 = 200              sample_std = 15
n = 40          alpha = 0.05
t_stat = (sample_mean - mu_0) / (sample_std / math.sqrt(n))
df = n - 1
t_critical = stats.t.ppf(alpha, df)
p_value = stats.t.cdf(t_stat, df)
print(f"t-statistic: {t_stat:.4f}")
print(f"Critical t-value (one-tailed, α=0.05): {t_critical:.4f}")
print(f"p-value: {p_value:.4f}")
if t_stat < t_critical:
print("Reject NULL HYPOTHESIS?he new recipe has significantly fewer calories.")
else: print("Fail to reject the null hypothesis: No significant reduction in calories.")
import numpy as np
import matplotlib.pyplot as plt
x=np.linspace(-5,20,1000)
y=stats.t.pdf(x,df)
plt.figure(figsize=(10,5))
plt.plot(x,y,label=f"t_distribution(df={df})")
plt.fill_between(x,0,y,where=(x<=t_critical),color='red',alpha=0.3)
plt.axvline(t_stat,color='blue',linestyle='--',linewidth=2,label=f"observed t={t_stat:.2f}")
plt.title("one sample one-tailed t test")
plt.xlabel("t_value")           plt.ylabel("probability")
plt.legend()    plt.grid(True)          plt.show()
```